

CORE CONCEPTS

Explore the context window

An interactive simulation of how Claude Code’s context window fills during a session. See what loads automatically, what each file read costs, and when rules and hooks fire.

Claude Code’s context window holds everything Claude knows about your session: your instructions, the files it reads, its own responses, and content that never appears in your terminal. The timeline below plays a full session from startup to compaction: what loads before you type, what each file read, rule, and hook adds as Claude works, and how a subagent keeps large reads out of your context. See [the written breakdown](#) for the same content as a list.

Explore the context window

A simulated session showing what enters context and what it costs

System

CLAUDE.md

Memory

Skills

MCP

Rules

You

Files

Output

Claude

Hooks

👁

= appears in your terminal

\$ c laude

▶ Start session

Watch what loads into context, from the moment you run `c laude` through a full conversation.

👁

Hover or click any event

Hover to preview. Click to pin so you can scroll.

KEY TAKEAWAY

A lot loads before you type anything. CLAUDE.md, memory, skills, and MCP tools are all in context before your first prompt.

IN YOUR TERMINAL YOU SEE

The input box, waiting for your first message. Everything above loads silently before you type anything.

What the timeline shows

The session walks through a realistic flow with representative token counts:

- **Before you type anything:** CLAUDE.md, auto memory, MCP tool names, and skill descriptions all load into context. Your own setup may add more here, like an `output style` or text from `--append-system-prompt`, which both go into the system prompt the same way.
- **As Claude works:** each file read adds to context, `path-scoped rules` load automatically alongside matching files, and a `PostToolUse hook` fires after each edit.
- **The follow-up prompt:** a `subagent` handles the research in its own separate context window, so the large file reads stay out of yours. Only the summary and a small metadata trailer come back.
- **At the end:** `/compact` replaces the conversation with a structured summary. Most startup content reloads automatically; the table below shows what happens to each mechanism.

What survives compaction

When a long session compacts, Claude Code summarizes the conversation history to fit the context window. What happens to your instructions depends on how they were loaded:

Mechanism	After compaction
System prompt and output style	Unchanged; not part of message history
Project-root CLAUDE.md and unscoped rules	Re-injected from disk
Auto memory	Re-injected from disk
Rules with <code>paths:</code> frontmatter	Lost until a matching file is read again
Nested CLAUDE.md in subdirectories	Lost until a file in that subdirectory is read again
Invoked skill bodies	Re-injected, capped at 5,000 tokens per skill and 25,000 tokens total; oldest dropped first
Hooks	Not applicable; hooks run as code, not context

Path-scoped rules and nested CLAUDE.md files load into message history when their trigger file is read, so compaction summarizes them away with everything else. They reload the next time Claude

reads a matching file. If a rule must persist across compaction, drop the `paths:` frontmatter or move it to the project-root `CLAUDE.md`.

Skill bodies are re-injected after compaction, but large skills are truncated to fit the per-skill cap, and the oldest invoked skills are dropped once the total budget is exceeded. Truncation keeps the start of the file, so put the most important instructions near the top of `SKILL.md`.

When your context fills up

Claude Code compacts automatically as you approach the limit, so a full context window doesn't end your session. The automatic pass works the same way as the `/compact` step in the timeline. See [When context fills up](#) for what it preserves.

You can also act before the automatic pass runs:

- **Compact with a focus:** run `/compact` with instructions, like `/compact focus on the auth bug fix`, before starting a long new task. The summary keeps what you choose instead of what the automatic pass guesses is important.
- **Clear between tasks:** run `/clear` when switching to unrelated work. Old conversation crowds out the files you need next and costs tokens on every message.
- **Delegate large reads:** send research to a [subagent](#) so the file contents stay in its context window, not yours.

If you need a larger window rather than a smaller conversation, Fable 5, Opus 4.6 and later, and Sonnet 4.6 support a 1 million token context window. See [Extended context](#) for availability by plan and how to select a `[1m]` model variant. Compaction works the same way at the larger limit.

Check your own session

The visualization uses representative numbers. To see your actual context usage at any point, run `/context` for a live breakdown by category with optimization suggestions. Run `/memory` to check which `CLAUDE.md` and auto memory files loaded at startup.

Related resources

For deeper coverage of the features shown in the timeline, see these pages:

- [Extend Claude Code](#): when to use `CLAUDE.md` vs skills vs rules vs hooks vs MCP

- **Store instructions and memories:** CLAUDE.md hierarchy and auto memory
- **Subagents:** delegate research to a separate context window
- **Best practices:** managing context as your primary constraint
- **Prompt caching:** which actions invalidate the cached prefix
- **Reduce token usage:** strategies for keeping context usage low